Day-8 PM Take-Home - Assignment

Part A:- Concept Application

File Link:-
https://github.com/NiranjanHebli/Day8-PM-Assignment/blob/main/password_analyzer.py

Output Screenshots:-

```
(base) niranjanhebli@Niranjans-MacBook-Air-117 Day8-PM-Assignment % python3 password_analyzer.py

Password Analyzer & Generator
1. Analyze Password
2. Generate Secure Password
3. Exit
Enter your choice (1/2/3): 1
Enter password:
>> Strength: 1/7 (Weak)
>> Missing: too short, uppercase, lowercase, digit, special char
>> Try again...

Enter password: p
>> Strength: 2/7 (Weak)
>> Missing: too short, uppercase, digit, special char
>> Try again...

Enter password: Pass
>> Strength: 3/7 (Medium)
>> Missing: too short, digit, special char
>> Try again...

Enter password: Password
>> Strength: 4/7 (Medium)
>> Missing: digit, special char
>> Try again...

Enter password: Password123
>> Strength: 5/7 (Strong)
>> Missing: special char
>> Password accepted!
```

```
(base) niranjanhebli@Niranjans-MacBook-Air-117 Day8-PM-Assignment % python3 password_analyzer.py

Password Analyzer & Generator
1. Analyze Password
2. Generate Secure Password
3. Exit
Enter your choice (1/2/3): 1
Enter password: Password@123
>> Strength: 7/7 (Very Strong)
>> Password accepted!

Password Analyzer & Generator
1. Analyze Password
2. Generate Secure Password
3. Exit
Enter your choice (1/2/3): 2
Enter desired password length: 10

Generated Password: YeNS2>r\qb
Strength: 5/7 (Strong)
Missing criteria: special char
Press Enter to return to menu...
```

Part B:- Interview Ready

Q1. Conceptual

Difference Between break and continue

The break statement immediately terminates the loop and transfers control to the statement following the loop. In contrast, the continue statement skips the current iteration and proceeds with the next iteration of the loop.

Example:

break:

```
for i in range(1, 6):
  if i == 3:
      break
  print(i)
```

This stops the loop when i becomes 3.

continue:-

```
for i in range(1, 6):
  if i == 3:
      continue
  print(i)
```

This skips printing 3 but continues the loop.

The else Clause in Loops

In Python, for and while loops can have an else clause. The else block executes only if the loop completes normally without encountering a break statement.

Example:

```
for num in [10, 20, 30]:
  if num == 25:
      break
else:
  print("Not found")
```

Here, since no break occurs, the else block executes.

A practical use case is search operations, where the else clause cleanly handles the "not found" case without requiring additional flags.

Q2. Coding

Brute Force Approach: O(n²) Using Nested Loops

This approach checks every possible pair in the list using two loops:-

File Link:-
https://github.com/NiranjanHebli/Day8-PM-Assignment/blob/main/find_pairs/brute_force.py

Code:-

```
def find_pairs(numbers, target):
    pairs = []

    for i in range(len(numbers)):
        for j in range(i + 1, len(numbers)):
            if numbers[i] + numbers[j] == target:
                pairs.append((numbers[i], numbers[j]))

    return pairs
```

Optimized Approach: O(n) Using a Set

We can reduce the time complexity by using a set to track visited numbers:-

File Link:-
https://github.com/NiranjanHebli/Day8-PM-Assignment/blob/main/find_pairs/optimized.py

Code:-

```
def find_pairs_optimized(numbers, target):
    seen = set()
    pairs = []

    for num in numbers:
        complement = target - num
```

```
        if(complement in seen):
            pairs.append((complement, num))

        seen.add(num)

    return pairs
```

Output Screenshot:-

```
● (base) niranjanhebli@Niranjans-MacBook-Air-117 Day8-PM-Assignment % cd find_pairs
● (base) niranjanhebli@Niranjans-MacBook-Air-117 find_pairs % python3 brute_force.py
  [(1, 5), (2, 4)]
● (base) niranjanhebli@Niranjans-MacBook-Air-117 find_pairs % python3 optimized.py
  [(2, 4), (1, 5)]
○ (base) niranjanhebli@Niranjans-MacBook-Air-117 find_pairs %
```

Q3. Debug and Analyze

There is no correctness bug for positive integers ≥ 2, the function correctly identifies prime numbers.

However, the loop runs from 2 to n-1, which is unnecessarily large and inefficient.

The main issue is inefficiency.

The loop checks all numbers from 2 up to n-1.
This results in O(n) time complexity.

But mathematically:

> If a number n has a factor greater than root(n), it must also have a factor smaller than root(n).

Therefore, we only need to check divisors up to root(n), not up to n-1

Optimized Code:-

```
def is_prime(n):
    if n < 2:
        return False

    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False

    return True
```

Part C:- AI-Augmented Task

AI Tool:- ChatGPT

Prompt:

Write a Python program that prints a diamond pattern made of asterisks (*). The user should input the number of rows for the upper half of the diamond. The program must properly align the pattern using spaces so that it appears symmetrical.

Requirements:

- Use only nested loops to construct the pattern.
- Do not use string multiplication (e.g., "*" * n or " " * n).
- Print the upper half and lower half separately.
- Ensure correct spacing so the diamond is center-aligned.
- Handle valid positive integer input only.

The AI's Output:-https://github.com/NiranjanHebli/Day8-PM-Assignment/blob/main/diamond_pattern/diamond_pattern.py

Critical Evaluation

Spacing Correctness

The spacing is correct. The program prints (rows - i) spaces before the stars, which keeps the diamond properly centered and symmetrical.

Readability

The code is clear and easy to understand. The upper and lower halves are separated logically, and variable names are meaningful.

Edge Cases (n = 0, n = 1)

- For n = 0, the program correctly prevents printing and shows a message.
- For n = 1, it correctly prints a single * without duplication.

Nested Loops vs String Tricks

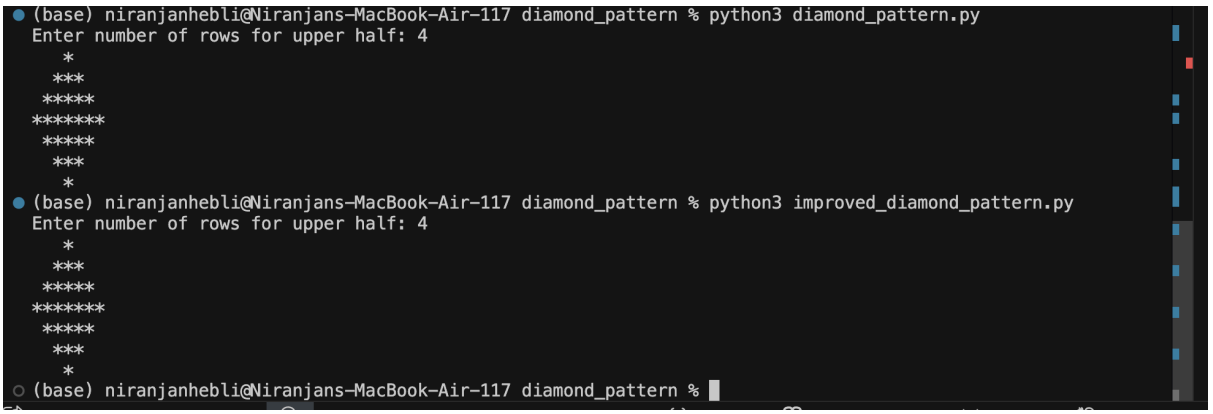
Using nested loops follows the requirement and clearly shows how the pattern is built. String multiplication would be shorter, but loops demonstrate better understanding.

Time Complexity

The time complexity is **O(n^2)** because the program prints roughly n^2 characters. This is expected and appropriate for pattern printing.

Improved Version:- https://github.com/NiranjanHebli/Day8-PM-Assignment/blob/main/diamond_pattern/improved_diamond_pattern.py

Output Screenshot:-



Part D:- Daily Transaction Analyzer

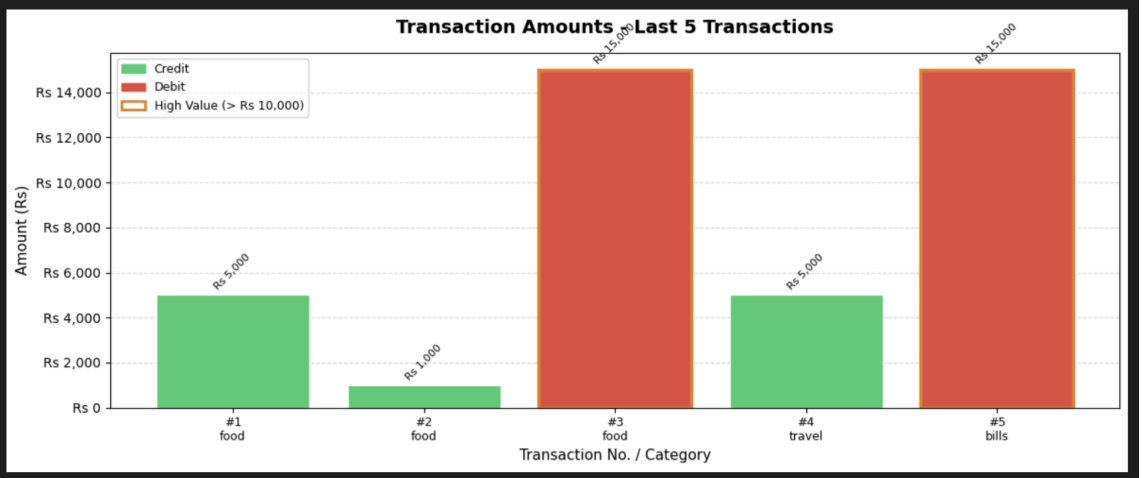
Notebook Link:-
https://github.com/NiranjanHebli/Day8-PM-Assignment/blob/main/transaction_analyzer.ipynb

Output Screenshots:-

```python
legend_handles = [
    mpatches.Patch(color=COLOR_CREDIT, label='Credit'),
    mpatches.Patch(color=COLOR_DEBIT,  label='Debit'),
    mpatches.Patch(facecolor='white', edgecolor=COLOR_HV, linewidth=2, label='High Value (> Rs 10,000)'),
]
ax.legend(handles=legend_handles, fontsize=9)

plt.tight_layout()
plt.show()
```

[29] Python

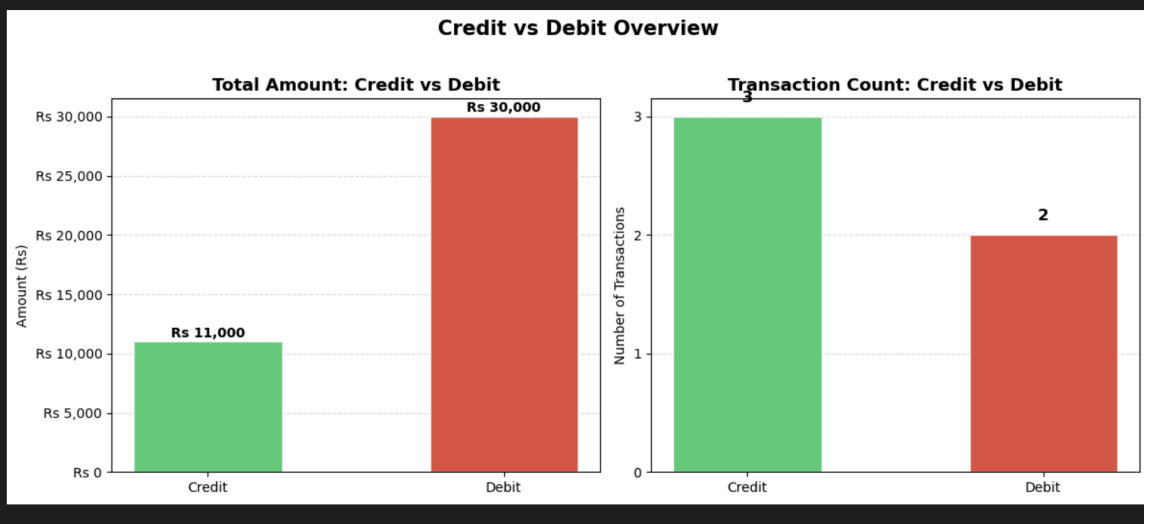


7. Chart 2 - Credit vs Debit Comparison (Grouped Bar Chart)

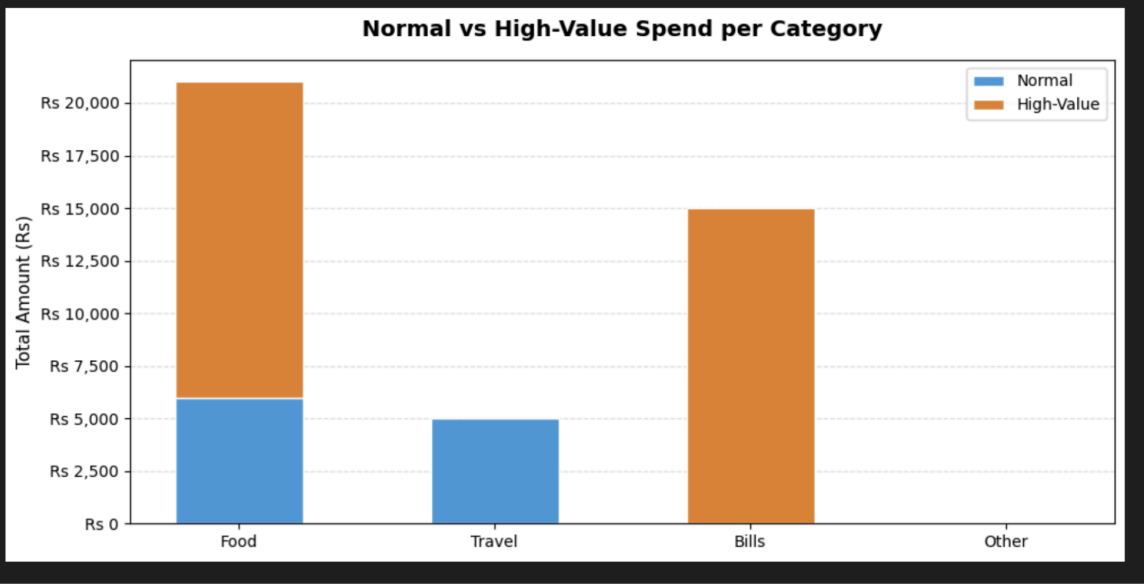
```python
                str(val), ha='center', va='bottom', fontsize=12, fontweight='bold')
ax2.set_title('Transaction Count: Credit vs Debit', fontsize=13, fontweight='bold')
ax2.set_ylabel('Number of Transactions')
ax2.yaxis.set_major_locator(plt.MaxNLocator(integer=True))
ax2.grid(axis='y', linestyle='--', alpha=0.4)
ax2.set_axisbelow(True)

plt.suptitle('Credit vs Debit Overview', fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

[30] Pytho




```python
ax.set_ylabel('Total Amount (Rs)', fontsize=11)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'Rs {x:,.0f}'))
ax.grid(axis='y', linestyle='--', alpha=0.4)
ax.set_axisbelow(True)
ax.legend(fontsize=10)

plt.tight_layout()
plt.show()
```

[32] Python